
Logistic Regression Implementation and Analysis

Cher Li

Department of Computer Engineering
McGill University
cher.li@mail.mcgill.ca

Nara Yun

Department of Software Engineering
McGill University
nara.yun@mail.mcgill.ca

Karen Cherry Chen Lai

Department of Computer Engineering
McGill University
karen.chenlai@mail.mcgill.ca

Abstract

This project investigates the performance of a logistic regression model on two benchmark datasets: the Chronic Kidney Disease (CKD) dataset and the Battery dataset. Logistic regression and K fold cross validation are implemented from scratch, alongside alterations in the type of model regularization. The impact of varying learning rates, feature selection, and dataset-specific transformations on model accuracy and generalization is explored. The results indicate that logistic regression achieves better accuracy and faster convergence with an optimized learning rate and number of epochs. Performance is enhanced through removing features based on their correlation with the target variable and the logarithm of individual features. These findings emphasize the importance of hyperparameter tuning and data preprocessing in developing effective linear classifiers.

1 Introduction

Machine Learning (ML) has been widely applied in various fields, including healthcare and technology. In this study, we explore the application of logistic regression, a foundational linear classification method, to forecast outcomes using two distinct datasets: the Chronic Kidney Disease (CKD) dataset and the Battery dataset. Logistic regression, a prevalent supervised learning algorithm, is employed for binary classification tasks, wherein it estimates the likelihood of an instance being associated with a particular class.

For this project, two datasets are provided for statistical analysis and histogram visualization. Linear classifier logistic regression is used as a classification algorithm on the two datasets. This algorithm is coded from scratch using Python on Google Colab alongside libraries such as Numpy and Pandas. The accuracy of the model is assessed through comparisons between predicted outputs and actual test labels. Additionally, k-fold cross-validation is applied to ensure robust performance evaluation and reduce variability in results. The goal is to correctly classify test data into distinct classes.

Beyond the initial model training, we explore methods to enhance classification accuracy. This includes tuning the learning rate and number of epochs to optimize convergence speed and accuracy, as well as investigating feature selection techniques. Specifically, adding or removing certain features and how they impact model generalization and decision boundaries is analyzed: the greatest accuracy results from taking the logarithm of a feature based off of known experiments.

Through a systematic evaluation of these factors, this study emphasizes the significance of hyperparameter tuning, feature engineering, and cross-validation in enhancing the performance of logistic regression models for practical, real-world applications.

2 Datasets

2.1 Data Overview

There are two datasets to analyze: one is the Chronic Kidney Disease (CKD) dataset, which consists of 330 patient samples, 28 numerical features, and a target binary classification of Normal or CKD. The features consist of biological measurements, such as creatinine levels, glomerular filtration rate (GFR), and urine protein levels.

The second is the Battery dataset, consisting of 732 battery samples, 32 numerical features, and a target binary classification of Normal or Defective. The features are battery attributes, such as batch number, internal resistance, and nominal voltage.

2.2 Data Analysis

After using Pandas to load the two datasets, statistical analysis is done on the data. For both datasets, the classification label, the feature mean and standard deviation, and the correlation between feature and label is plotted using histograms and aid from [1]. For example, figures 1 and 2 show the dataset class label distribution. The count of Normal vs CKD or Defective labels is compared with each other, showing an even number of either labels.

Likewise, figures 3 and 4 show the distribution of the feature mean after the data undergoes normalization of z-standardization taken from [2]. Positive values denote features with an average above the standardized zero mean, while negative values denote ones with an average below. Figures 5 and 6 represent the data standard deviation for the raw data, showing the scale of the different features. If standardized, the standard deviation values would be equal across different features and wouldn't convey much information.

Finally, the correlation between features and the classification label is assessed in figures 7 and 8. CKD and Defective are denoted by 1, so the features more likely to be correlated with class 1 have a greater absolute value. For example, feature 2 for both datasets has a high correlation, which means that an increase in feature 2 is positively correlated with CKD and Defective respectively.

2.3 Logistic Regression Implementation

For the logistic regression algorithm, regularization model, penalty, learning rate, number of epochs, and weights are attributes of the logistic regression class. A sigmoid function returns the result given $z = \text{linear model of the different weights}$.

There are three models: default, L1 regularization, and L2 regularization, so the loss function takes into account self.regularization and outputs the sum between loss1 and loss2. Variable loss1 represents the default cross entropy, and variable loss2 represents the additional loss from either L1 or L2 regularization. L1 and L2 regularization are coded based off of the lecture slides and [3].

The function gradient descent computes the gradient of the loss function, taking into account which regularization model is inputted. Fit initializes the weight vector, with an extra term accounting for the bias, and looping over the number of epochs, the function computes the linear model before applying the sigmoid and gradient descent functions. Using the gradient, self.weights is updated to reach a more suitable solution. Finally, the predict function takes a set of test data X and outputs the predicted binary output using the sigmoid function. When coding this function, [4] is also referenced as a guideline.

3 Results

3.1 Cross Validation

K-fold cross validation is implemented from scratch as a Python class; it takes in X (test features) and the corresponding y (test labels) to validate the accuracy of the input model. After splitting the data into k folds, each fold will serve as the validation set for that particular instance of cross validation. The remaining folds are combined into one training set (train idx), and the model is trained using the fit function.

Utilizing the model's predictions, another function called `Accu_eval` is created to compute the accuracy between predicted and actual label values. A separate function is also implemented to centralize evaluation of metrics such as precision and recall from the confusion matrix. The lecture slides and [5] are referenced for this. Finally, the k-fold cross validation returns the average of the computed accuracies.

3.2 Model Alterations

After setting up cross validation, hyperparameters such as learning rate as well as feature addition are adjusted to aid in accuracy change analysis. The following experiments are tested on the default logistic regression model, L1 regularization, and L2 regularization. Training parameters are researched using [6].

3.2.1 Impact of Learning Rate on Model

The learning rate significantly influences the performance of logistic regression. As illustrated in Figure 9 and Table 1, increasing the learning rate improves accuracy for both datasets at the beginning, but beyond a certain point, the gains diminish. The Battery dataset achieves over 85% accuracy with an optimal learning rate around 0.2, showing a rapid convergence. In contrast, the CKD dataset improves more gradually, stabilizing near 65% accuracy at higher learning rates. This suggests that CKD data may require additional preprocessing or feature selection. Overall, tuning the learning rate is essential, as moderate values (0.2–0.3) balance convergence speed and accuracy, preventing slow learning or instability in gradient descent updates.

Table 1: Learning Rate Accuracy Results

Data	Logistic Regression	L1 Regularization	L2 Regularization	Learning Rate
CKD	0.5121	0.4364	0.4879	0.01
CKD	0.5636	0.4394	0.4939	0.05
CKD	0.6182	0.4758	0.4939	0.1
CKD	0.6545	0.4697	0.4939	0.5
Battery	0.6845	0.4577	0.6408	0.01
Battery	0.8128	0.4522	0.6709	0.05
Battery	0.8361	0.4509	0.6763	0.1
Battery	0.8661	0.5109	0.6777	0.5

3.2.2 Impact of Number of Epochs on Model

Similar to the learning rate, the number of epochs significantly influences the performance. As shown in Figure 10 and Table 2, increasing the number of epochs improves accuracy for both datasets, but the gains diminish beyond a specific number of epochs. The CKD dataset improves more gradually compared to the Battery dataset, reaching an accuracy of 85% and 63% with an optimal number of epoch of 1000. Overall, adjusting the number of epochs is vital to balance convergence speed versus accuracy.

Also, completing the k-fold cross validation for both datasets over the range of epochs has different run times. It takes 25.7 seconds for the default model, 40.5 seconds for L1 regularization, and 51.1 seconds for L2 regularization. This shows that for these datasets, the default model is the most efficient without compromising on accuracy. The model for future experiments is 1000 epochs for a learning rate of 0.3: the CKD dataset has an accuracy of 0.6515 and the battery dataset has an accuracy of 0.8552.

Table 2: # Epochs Accuracy Results

Data	Logistic Regression	L1 Regularization	L2 Regularization	# Epochs
CKD	0.5121	0.4606	0.4879	100
CKD	0.5636	0.4273	0.4939	500
CKD	0.6182	0.4758	0.4939	1000
CKD	0.6333	0.4424	0.4939	2000
CKD	0.6545	0.4303	0.4939	5000
Battery	0.6845	0.4714	0.6421	100
Battery	0.6845	0.4822	0.6709	500
Battery	0.8361	0.4509	0.6763	1000
Battery	0.8470	0.5111	0.6777	2000
Battery	0.8661	0.4769	0.6777	5000

3.2.3 Impact of K Values on Model

The default value for k fold cross validation is $k = 10$, and even though altering the value of k doesn't influence the accuracy as much as learning rate or the number of epochs, the trends are still viable for discussion. As shown in Figure 11 and Table 3, when $k = 8$, the CKD accuracy for the default model is higher than if k is altered (an increase of 0.06).

A similar thing occurs for the L2 model, where $k = 10$ yields the best accuracy. Only L1 regularization follows a general trend of a higher k value equating to higher accuracy. The range of K values does not affect the accuracy for the battery dataset as much as the CKD dataset, demonstrating dataset size and complexity difference. Also, the run time for $k = 10$ on both datasets is 8.02 seconds, and it's 4.41 seconds for $k = 6$, showing the trade off between efficiency and accuracy.

Table 3: K Values Accuracy Results

Data	Logistic Regression	L1 Regularization	L2 Regularization	K Values
CKD	0.6636	0.4485	0.4758	6
CKD	0.6907	0.4578	0.4848	8
CKD	0.6515	0.4424	0.4939	10
CKD	0.6455	0.4697	0.4788	15
Battery	0.8484	0.4918	0.6612	6
Battery	0.8538	0.4986	0.6679	8
Battery	0.8552	0.5191	0.6777	10
Battery	0.8567	0.4946	0.6776	15

3.2.4 Impact of Feature Augmentation on Model

A range of feature augmentation techniques are utilized to discover the best accuracy. These techniques are applied to individual features one at a time, meaning there are improvements for further studies. For this section, only the default model is trained and tested upon for consistency, as shown in Table 4.

Some feature changes that increase the accuracy for one of the datasets include taking the logarithm and taking the square root, while a change that decreases the accuracy is taking the reciprocal of a feature. This could be because of data skewness and variance, while the reciprocal increases sensitivity to values closer to 0.

Also, a feature subset through removing features highly correlated based on a certain threshold does not significantly improve the performance of the model. This helps to demonstrate that despite the correlation, different features bestow unique characteristics upon the data.

Table 4: Feature Addition / Removal Average 10-Fold Cross Validation Accuracy

Type	Battery	CKD
Feature removal	0.8552	0.6515
Squaring	0.8517	0.6525
Cubing	0.8538	0.6455
Logarithm	0.8602	0.6515
Square root	0.8502	0.6535
Reciprocal	0.7604	0.6232
Pair Multiplication	0.8575	0.6414
Sine	0.8538	0.6505

4 Discussion and Conclusion

After implementing the logistic regression and k-fold cross validation code from scratch before adjusting various parameters to increase model accuracy, the model balancing learning convergence and accuracy involves a learning rate of 0.3, number of epochs of 1000, and a feature alteration through taking the logarithm of a feature. As shown in Figure 12, adding the log of feature 17 gives an accuracy of 0.6697 for the CKD dataset, and feature 16 gives 0.8648 for the battery dataset, an increase of 0.03 and 0.01 respectively.

Future investigations include further data analysis and preprocessing such as plotting pairplots instead of only histograms as is recommended. Experimentation with more variety of models and linear combination of feature augmentation are also possible changes to explore.

Key takeaways from this project are understanding logistic regression and k fold cross validation in order to implement them from scratch, the importance of tuning hyperparameters like learning rates to balance efficiency and accuracy, and how different datasets involve different model tuning and feature augmentation due to differences in size and distribution. There are also broader impacts: machine learning algorithms like logistic regression are able to offer a method for classifying important information for industries like the medical field, but the ethical consequence of energy consumption and personal biases should also be taken into consideration.

Statement of Contributions

This project is a team effort and every team member has contributed an equal amount of time to bring the results to completion. Each team member has also aided in debugging and report writing.

Nara Yun: Data analysis and logistic regression function

Cher Li: Hyperparameter adjustments and graphing

Karen Cherry Chen Lai: Accuracy evaluation and Z normalization

References

- [1] matplotlib, "Pyplot tutorial — Matplotlib 3.8.0 documentation," matplotlib.org, 2024. <https://matplotlib.org/stable/tutorials/pyplot.html>
- [2] Codecademy Team, "Normalization," Codecademy. <https://www.codecademy.com/article/normalization>
- [3] "L1 and L2 Regularization Methods, Explained | Built In," builtin.com. <https://builtin.com/data-science/l2-regularization>
- [4] "Gradient Descent in Linear Regression," GeeksforGeeks, Sep. 13, 2018. <https://www.geeksforgeeks.org/gradient-descent-in-linear-regression/>
- [5] IBM, "Confusion matrix," Ibm.com, Jan. 19, 2024. <https://www.ibm.com/think/topics/confusion-matrix>
- [6] D. Wei, "Understanding Parameters in ML Training — Batch Size, Iteration, Epoch, Learning Rate," Medium, Jan. 28, 2024. <https://medium.com/@weidagang/understanding-parameters-in-ml-training-batch-size-iteration-epoch-learning-rate-a1217d8f80e1>

Appendix

4.1 Appendix A: Data Visualization

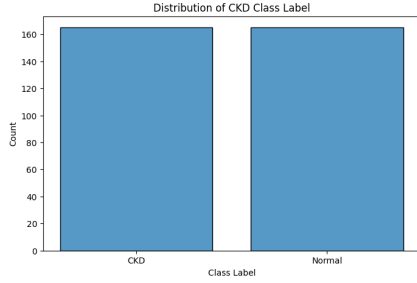


Figure 1: CKD vs Normal Distribution

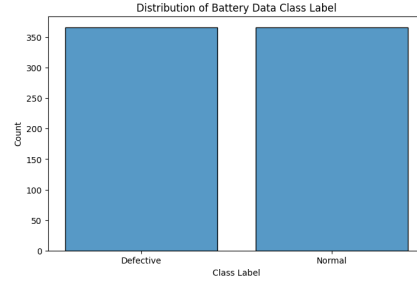


Figure 2: Defective vs Normal

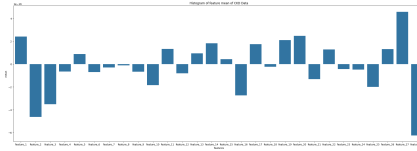


Figure 3: Feature Mean of CKD

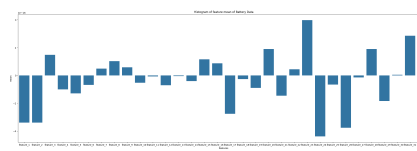


Figure 4: Feature Mean of Batteries

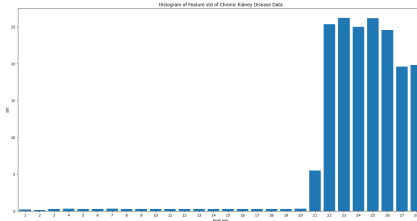


Figure 5: CKD Standard Deviation

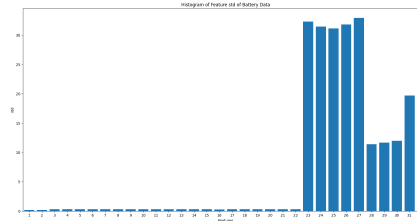


Figure 6: Battery Standard Deviation

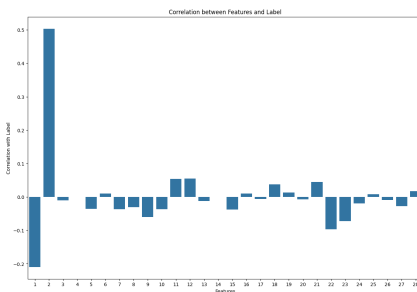


Figure 7: CKD Feature Correlation

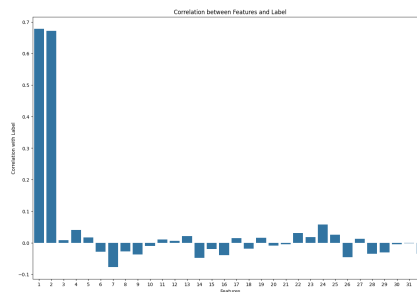


Figure 8: Battery Feature Correlation

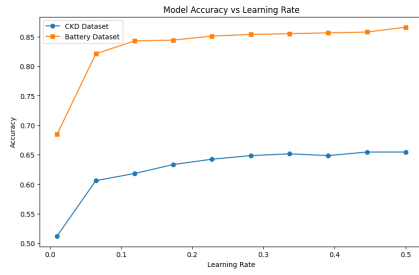


Figure 9: Accuracy vs Learning Rate

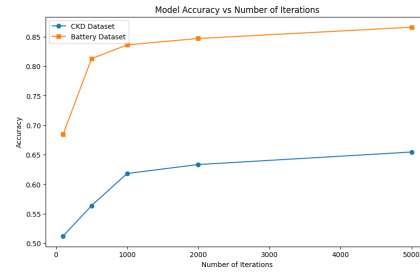


Figure 10: Accuracy vs # Epochs

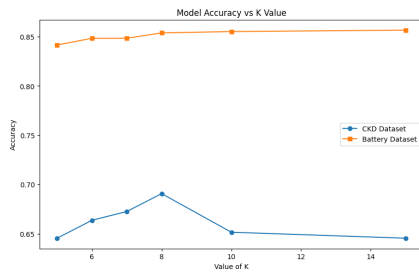


Figure 11: Accuracy vs K Value

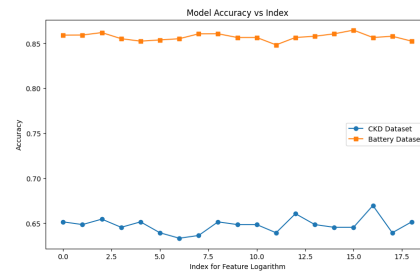


Figure 12: Accuracy vs Log of Features

4.2 Appendix B: Code

✓ ReadMe:

1. Download the 2 csv files
2. Create a folder called ECSE551 on google drive
3. Upload the csv files in the folder
4. Go to the Runtime tab and click Run all

```
[ ] import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import time
import sys
from google.colab import drive
drive.mount('/content/gdrive')
```

~ Loading the data

```
[ ] # built in runtime of cells
    !pip install ipython-autotime
    %load_ext autotime

    # Load Chronic Kidney Disease data
    ckd_data = pd.read_csv('/content/gdrive/MyDrive/ECSE551/CKD.csv')
    ckd_data_df = ckd_data.iloc[:,1:29]
    #print(ckd_data_df.head(2))
    ckd_class_df = ckd_data['label']
    #print(ckd_class_df.head(2))

    # Load battery dataset data
    battery_data = pd.read_csv('/content/gdrive/MyDrive/ECSE551/Battery_Dataset.csv')
    battery_data_df = battery_data.iloc[:,1:33]
    #print(battery_data_df.head(2))
    battery_class_df = battery_data['label']
    #print(battery_class_df.head(2))

    # observe the overall data stats
    print(ckd_data_df.describe())
    # observe the counts of each class
    print(ckd_class_df.value_counts())
```

```
standardized_ckd_data = ckd_data_df.iloc[:,1:29]
mean = standardized_ckd_data.mean()
std = standardized_ckd_data.std()
standardized_ckd_data = (standardized_ckd_data - mean) / std
standardized_ckd_data = standardized_ckd_data.to_numpy()
print(type(standardized_ckd_data))
print(standardized_ckd_data.shape)
print(standardized_ckd_data)
```

```
standardized_battery_data = battery_data_df.iloc[:,1:33]
mean = standardized_battery_data.mean()
std = standardized_battery_data.std()
standardized_battery_data = (standardized_battery_data - mean) / std
standardized_battery_data = standardized_battery_data.to_numpy()
print(type(standardized_battery_data))
print(standardized_battery_data.shape)
print(standardized_battery_data)
```

~ Chronic Kidney Disease (CKD) Data Features Distribution Visualization

```
[ ] # Plot of class distribution
    plt.figure(figsize=(8,5))
    sns.histplot(ckd_class_df, kde=False, bins=2, shrink=0.8)
    plt.xticks(np.arange(0,2,step=1))
    plt.title("Distribution of CKD Class Label")
    plt.xlabel("Class Label")
    plt.ylabel("Count")
    plt.show()
```

```
• # Plot mean value of raw datas
    plt.figure(figsize=(20,10))
    #print(ckd_data_df.iloc[:,1:29].mean())

    # Calculate mean values of the features
    means = ckd_data_df.iloc[:,1:29].mean()
    x_ticks = range(1, len(means)+1)
    plt.bar(x_ticks, means.values)

    plt.title("Histogram of Feature Means of Chronic Kidney Disease Data")
    plt.xlabel("Features")
    plt.ylabel("Mean")
    plt.xticks(x_ticks)

    # Adjust x-axis limits to remove extra blank space
    plt.xlim(0.5, len(means) + 0.5)
    plt.show()
```

```
# Plot standard deviation value of raw datas
plt.figure(figsize=(20, 10))
#print(ckd_data_df.iloc[:,1:29].std())
std = ckd_data_df.iloc[:,1:29].std()
x_ticks = range(1, len(std)+1)
plt.bar(x_ticks, std.values)

plt.title("Histogram of Feature std of Chronic Kidney Disease Data")
plt.xlabel("Features")
plt.ylabel("std")
plt.xticks(x_ticks)

# Adjust x-axis limits to remove extra blank space
plt.xlim(0.5, len(std) + 0.5)
plt.show()
```

```
ckd_class_df = ckd_class_df.map({'Normal': 0, 'CKD': 1})

# print(ckd_data) # ID, features, label
# print(ckd_data_df) # no ID

#Plot mean values of standardized datas
plt.figure(figsize=(30, 10))
means = standardized_ckd_data.mean(axis=0)
feature_names = ckd_data_df.columns[1:]
print(means)
sns.barplot(x=feature_names, y=means)
plt.title("Histogram of feature mean of CKD Data")
plt.xlabel("features")
plt.ylabel("mean")
plt.show()
```

```
# Plot the correlation of each feature and the label
CKD_correlation_values = None
ckd_data_df = ckd_data.iloc[:, 1:29] # correlaion matrix
ckd_data_df['label'] = ckd_class_df
corr_matrix = ckd_data_df.corr()
feature_corr = corr_matrix['label'].drop('label') # Exclude self-correlation

selected_features = feature_corr

CKD_correlation_values = feature_corr.tolist() # record the correlations down

feature_corr.index = [str(i+1) for i in range(len(feature_corr.index))]
selected_features.index = [str(i+1) for i in range(len(selected_features.index))]
print(feature_corr)

plt.figure(figsize=(15, 10))
sns.barplot(x=selected_features.index, y=selected_features.values)
plt.title('Correlation between Features and Label')
plt.xlabel('Features')
plt.ylabel('Correlation with Label')
plt.show()
```

▼ Battery Data Features Distribution Visualization

```
[ ] print(battery_data_df.describe())
print(battery_class_df.value_counts())
```

```
• # Plot of class distribution
plt.figure(figsize=(8,5))
sns.histplot(battery_class_df, kde = False, bins = 2, shrink=0.8)
plt.xticks(np.arange(0,2,step=1))
plt.title("Distribution of Battery Data Class Label")
plt.xlabel("Class Label")
plt.ylabel("Count")
plt.show()
```

```
[ ] # Plot mean value of raw datas
plt.figure(figsize=(20,10))
#print(battery_data_df.iloc[:,1:33].mean())

# Calculate mean values of the features
means = battery_data_df.iloc[:,1:33].mean()
x_ticks = range(1, len(means)+1)
plt.bar(x_ticks, means.values)

plt.title("Histogram of Feature Means of Battery Data")
plt.xlabel("Features")
plt.ylabel("Mean")
plt.xticks(x_ticks)

# Adjust x-axis limits to remove extra blank space
plt.xlim(0.5, len(means) + 0.5)
plt.show()
```

```
# Plot standard deviation value of raw datas
plt.figure(figsize=(20, 10))
#print(battery_data_df.iloc[:,1:33].std())
std = battery_data_df.iloc[:,1:33].std()
x_ticks = range(1, len(std)+1)
plt.bar(x_ticks, std.values)

plt.title("Histogram of Feature std of Battery Data")
plt.xlabel("Features")
plt.ylabel("std")
plt.xticks(x_ticks)

# Adjust x-axis limits to remove extra blank space
plt.xlim(0.5, len(std) + 0.5)
plt.show()
```

```
battery_class_df = battery_class_df.map({'Normal': 0, 'Defective': 1})
#Plot mean values of standardized datas
plt.figure(figsize=(32, 10))
means = standardized_battery_data.mean(axis=0)
feature_names = battery_data_df.columns[1:]
print(means)
sns.barplot(x=feature_names, y=means)
plt.title("Histogram of feature mean of Battery Data")
plt.xlabel("features")
plt.ylabel("mean")
plt.show()
```

```
#Plot standard deviation values of standardized datas
plt.figure(figsize=(30, 10))
std = standardized_battery_data.std(axis=0)
feature_names = battery_data_df.columns[1:]
print(std)
sns.barplot(x=feature_names, y=std)
plt.title("Histogram of feature standard deviations of Battery Data")
plt.xlabel("features")
plt.ylabel("std")
plt.show()
```

```

# Plot the correlation of each feature and the label
battery_data_correlation_values = None
battery_data_df = battery_data.iloc[:, 1:33]
battery_data_df['label'] = battery_class_df
corr_matrix = battery_data_df.corr()
feature_corr = corr_matrix['label'].drop('label') # Exclude self-correlation

selected_features = feature_corr

battery_data_correlation_values = feature_corr.tolist() # record the correlations down

feature_corr.index = [str(i+1) for i in range(len(feature_corr.index))]
selected_features.index = [str(i+1) for i in range(len(selected_features.index))]
print(feature_corr)

plt.figure(figsize=(15, 10))
sns.barplot(x=selected_features.index, y=selected_features.values)
plt.title('Correlation between Features and Label')
plt.xlabel('Features')
plt.ylabel('Correlation with Label')
plt.show()

```

Implementation of the Logistic Regression Algorithm

```

[ ] class LogisticRegression:
    def __init__(self, regularization=None, penalty=0.1, learning_rate=0.01, num_epochs=1000):
        self.regularization = regularization
        self.penalty = penalty
        self.learning_rate = learning_rate
        self.num_epochs = num_epochs
        self.weights = None

    def sigmoid(self, z):
        z = np.clip(z, -709, 709) # To avoid overflow
        return 1 / (1 + np.exp(-z))

    def loss(self, y_pred, y, W):
        y_pred = np.clip(y_pred, 1e-7, 1 - 1e-7)
        loss1 = -np.mean(y * np.log(y_pred) + (1 - y) * np.log(1 - y_pred))
        loss2 = 0
        if self.regularization == 'l1':
            loss2 = self.penalty * np.sum(np.abs(W[1:])) # Exclude bias term
        elif self.regularization == 'l2':
            loss2 = (self.penalty / 2) * np.sum(W[1:] ** 2) # Exclude bias term
        return loss1 + loss2

    def gradient_descent(self, X_bias, y_hat, y, W):
        grad1 = -X_bias.T.dot((y - y_hat)) / y.shape[0]
        grad2 = np.zeros_like(W)
        if self.regularization == 'l1':
            grad2[1:] = self.penalty * np.sign(W[1:]) # No regularization on bias
        elif self.regularization == 'l2':
            grad2[1:] = 2 * self.penalty * W[1:]
        return grad1 + grad2

```

```

def fit(self, X, y):
    n_samples, n_features = X.shape
    self.weights = np.zeros(n_features + 1) # Include bias
    X_bias = np.hstack((np.ones((n_samples, 1)), X))
    self.loss_history = []

    for _ in range(self.num_epochs):
        linear_model = np.dot(X_bias, self.weights)
        y_pred = self.sigmoid(linear_model)

        current_loss = self.loss(y_pred, y, self.weights)
        self.loss_history.append(current_loss)

        dw = self.gradient_descent(X_bias, y_pred, y, self.weights)
        self.weights -= self.learning_rate * dw

    def predict(self, X):
        n = X.shape[0]
        X_bias = np.hstack((np.ones((n, 1)), X))
        linear_model = np.dot(X_bias, self.weights) # No self.bias
        y_pred = self.sigmoid(linear_model)
        return (y_pred >= 0.5).astype(int)

    def get_training_time(self):
        return self.train_time

    def z_score_normalize(X, normalized_indices=None):
        X = X.astype(float) # Ensure data type consistency
        X_normalized = X.copy() # Copy the dataset to avoid modifying original data

        num_features = X.shape[1]

        for i in range(num_features):
            if normalized_indices is None or i not in normalized_indices:
                mean = np.mean(X[:, i])
                std = np.std(X[:, i])

                # Avoid division by zero if std is 0
                if std > 0:
                    X_normalized[:, i] = (X[:, i] - mean) / std

        return X_normalized

```

A function to evaluate the models' accuracy

```

[ ] def Accu_eval(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    accuracy = np.sum(y_true == y_pred) / len(y_true)
    return accuracy

```

v k-fold cross validation

```
[ ] def k_fold_validation(model, X, y, k=10):
    indices = np.arange(len(X))

    np.random.seed(42) # Set seed for reproducibility
    np.random.shuffle(indices)

    folds = np.array_split(indices, k) # Splitting data into K folds
    accuracies = []

    for i in range(k):
        val_idx = folds[i] # Current validation fold indices
        train_idx = np.hstack([folds[j] for j in range(k) if j != i]) # Train on remaining folds

        X_train, y_train = X[train_idx], y[train_idx]
        X_val, y_val = X[val_idx], y[val_idx]

        model.fit(X_train, y_train)
        predictions = model.predict(X_val)

        accuracy = Accu_eval(y_val, predictions)
        accuracies.append(accuracy)

    return np.mean(accuracies)
```

```
# Evaluating Metrics
def evaluate_metrics(y_true, y_pred):
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))

    accuracy = (TP + TN) / (TP + TN + FP + FN) if (TP + TN + FP + FN) > 0 else 0
    precision = TP / (TP + FP) if (TP + FP) > 0 else 0
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0
    false_positive_rate = FP / (FP + TN) if (FP + TN) > 0 else 0

    return {
        "TP": TP,
        "TN": TN,
        "FP": FP,
        "FN": FN,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "False Positive Rate": false_positive_rate
    }
```

```
# Divide into train and test datasets
def train_test_split(df, train_size=0.7, shuffle=True):
    if shuffle:
        df_shuffled = df.sample(frac=1, random_state=42)
    else:
        df_shuffled = df
    split = int(train_size * len(df))

    train_set = df_shuffled[:split]
    test_set = df_shuffled[split:]
    return train_set, test_set
```

```
# Initialize model
learning_rate = [0.01, 0.05, 0.1, 0.5]
model_types = ["default", "l1 regularization", "l2 regularization", "feature selection"]
model = LogisticRegression("l1", learning_rate=0.3, num_epochs=1000)

# Perform manual cross-validation
ckd_accuracy = k_fold_validation(model, ckd_data_df.values, ckd_class_df.values)
battery_accuracy = k_fold_validation(model, battery_data_df.values, battery_class_df.values)
print(f"CKD Dataset Accuracy: {ckd_accuracy:.4f}")
print(f"Battery Dataset Accuracy: {battery_accuracy:.4f}")

# graphs the accuracy over range of learning rates
test = {}
x = 0.01
while x < 0.5:
    model = LogisticRegression("l1", learning_rate=x, num_epochs=1000)
    accuracy = k_fold_validation(model, ckd_data_df.values, ckd_class_df.values)
    test[x] = accuracy
    x = x * 0.1
plt.plot(list(test.keys()), list(test.values()))
plt.title("Accuracy vs Learning Rate for CKD Dataset Graph")
plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.show()

# same for battery dataset
test = {}
x = 0.01
while x < 0.5:
    model = LogisticRegression("l1", learning_rate=x, num_epochs=1000)
    accuracy = k_fold_validation(model, battery_data_df.values, battery_class_df.values)
    test[x] = accuracy
    x = x * 0.1
plt.plot(list(test.keys()), list(test.values()))
plt.title("Accuracy vs Learning Rate for Battery Dataset Graph")
plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.show()
```

```
# Divide into train and test datasets
def train_test_split(df, train_size=0.7, shuffle=True):
    if shuffle:
        df_shuffled = df.sample(frac=1, random_state=42)
    else:
        df_shuffled = df
    split = int(train_size * len(df))

    train_set = df_shuffled[:split]
    test_set = df_shuffled[split:]
    return train_set, test_set

ckd_data['label'] = ckd_data['label'].map({'CKD': 1, 'Normal': 0})
train_ckd, test_ckd = train_test_split(ckd_data)
train_ckd_x = train_ckd.iloc[:, :-1].to_numpy()
train_ckd_y = train_ckd.iloc[:, -1].to_numpy()
test_ckd_x = test_ckd.iloc[:, :-1].to_numpy()
test_ckd_y = test_ckd.iloc[:, -1].to_numpy()
model = LogisticRegression("l1", learning_rate=0.3, num_epochs=1000)
accuracy = k_fold_validation(model, train_ckd_x, train_ckd_y)
print(f'CKD Dataset Accuracy after train test split: {accuracy:.4f}')

battery_data['label'] = battery_data['label'].map({'Defective': 1, 'Normal': 0})
train_bat, test_bat = train_test_split(battery_data)
train_bat_x = train_bat.iloc[:, :-1].to_numpy()
train_bat_y = train_bat.iloc[:, -1].to_numpy()
test_bat_x = test_bat.iloc[:, :-1].to_numpy()
test_bat_y = test_bat.iloc[:, -1].to_numpy()
model = LogisticRegression("l1", learning_rate=0.3, num_epochs=1000)
accuracy = k_fold_validation(model, train_bat_x, train_bat_y)
print(f'Battery Dataset Accuracy after train test split: {accuracy:.4f}')
```

▼ Standardizing the data

```
[ ] # Load data
ckd_data = pd.read_csv('/content/gdrive/MyDrive/ECSE551/CKD.csv')
battery_data = pd.read_csv('/content/gdrive/MyDrive/ECSE551/Battery_Dataset.csv')

# Extract features and labels
ckd_features = ckd_data.iloc[:, 1:29].copy()
ckd_labels = ckd_data['label'].map({'Normal': 0, 'CKD': 1})

battery_features = battery_data.iloc[:, 1:33].copy()
battery_labels = battery_data['label'].map({'Normal': 0, 'Defective': 1})

# Normalize specific feature ranges
def z_score_normalize(X, start, end):
    X = X.astype(float) # Ensure consistency
    for i in range(start, end):
        mean = np.mean(X[:, i])
        std = np.std(X[:, i])
        if std > 0:
            X[:, i] = (X[:, i] - mean) / std
    return X

ckd_features = ckd_features.to_numpy()
ckd_features = z_score_normalize(ckd_features, 20, 28)

battery_features = battery_features.to_numpy()
battery_features = z_score_normalize(battery_features, 22, 32)

ckd_X, ckd_y = ckd_features, ckd_labels.to_numpy()
battery_X, battery_y = battery_features, battery_labels.to_numpy()
```

```
# Initialize model
model = LogisticRegression(learning_rate=0.3, num_epochs=1000)

# Perform k-fold cross-validation
ckd_kfold_accuracy = k_fold_validation(model, ckd_X, ckd_y)
battery_kfold_accuracy = k_fold_validation(model, battery_X, battery_y)

# Train the model on the full dataset
model.fit(ckd_X, ckd_y)
ckd_predictions = model.predict(ckd_X)
ckd_accu_eval = Accu_eval(ckd_y, ckd_predictions)

model.fit(battery_X, battery_y)
battery_predictions = model.predict(battery_X)
battery_accu_eval = Accu_eval(battery_y, battery_predictions)

# Evaluate additional metrics
ckd_metrics = evaluate_metrics(ckd_y, ckd_predictions)
battery_metrics = evaluate_metrics(battery_y, battery_predictions)

print(f'CKD Dataset Accuracy (k-fold validation): {ckd_kfold_accuracy:.4f}')
print(f'CKD Dataset Accuracy (Accu_eval direct evaluation): {ckd_accu_eval:.4f}')
print(f'Battery Dataset Accuracy (k-fold validation): {battery_kfold_accuracy:.4f}')
print(f'Battery Dataset Accuracy (Accu_eval direct evaluation): {battery_accu_eval:.4f}')

# Print detailed evaluation metrics
print("\n==== CKD Dataset Metrics =====")
print(f'True Positives (TP): {ckd_metrics["TP"]}')
print(f'True Negatives (TN): {ckd_metrics["TN"]}')
print(f'False Positives (FP): {ckd_metrics["FP"]}')
print(f'False Negatives (FN): {ckd_metrics["FN"]}')
print(f'Precision: {ckd_metrics["Precision"]:.4f}')
print(f'Recall: {ckd_metrics["Recall"]:.4f}')
print(f'False Positive Rate: {ckd_metrics["False Positive Rate"]:.4f}')

print("\n==== Battery Dataset Metrics =====")
print(f'True Positives (TP): {battery_metrics["TP"]}')
print(f'True Negatives (TN): {battery_metrics["TN"]}')
print(f'False Positives (FP): {battery_metrics["FP"]}')
print(f'False Negatives (FN): {battery_metrics["FN"]}')
print(f'Precision: {battery_metrics["Precision"]:.4f}')
print(f'Recall: {battery_metrics["Recall"]:.4f}')
print(f'False Positive Rate: {battery_metrics["False Positive Rate"]:.4f}')
```

```
# graphs the accuracy over range of learning rates
learning_rates = np.linspace(0.01, 0.5, 10)
ckd_accuracies = []
battery_accuracies = []

for lr in learning_rates:
    model = LogisticRegression(learning_rate=lr, num_epochs=1000)

    ckd_accuracy = k_fold_validation(model, ckd_X, ckd_y)
    battery_accuracy = k_fold_validation(model, battery_X, battery_y)

    ckd_accuracies.append(ckd_accuracy)
    battery_accuracies.append(battery_accuracy)

# Plot results
plt.figure(figsize=(10, 6))
plt.plot(learning_rates, ckd_accuracies, marker='o', label="CKD Dataset")
plt.plot(learning_rates, battery_accuracies, marker='s', label="Battery Dataset")

plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.title("Model Accuracy vs Learning Rate")
plt.legend()
plt.grid(False)
plt.show()
```

```
# graphs accuracy over range of epochs
iterations = [100, 500, 1000, 2000, 5000]
ckd_accuracies = []
battery_accuracies = []

for num_epochs in iterations:
    model = LogisticRegression(learning_rate=0.1, num_epochs=num_epochs)

    ckd_accuracy = k_fold_validation(model, ckd_X, ckd_y)
    battery_accuracy = k_fold_validation(model, battery_X, battery_y)

    ckd_accuracies.append(ckd_accuracy)
    battery_accuracies.append(battery_accuracy)

# Plot results
plt.figure(figsize=(10, 6))
plt.plot(iterations, ckd_accuracies, marker='o', label="CKD Dataset")
plt.plot(iterations, battery_accuracies, marker='s', label="Battery Dataset")

plt.xlabel("Number of Iterations")
plt.ylabel("Accuracy")
plt.title("Model Accuracy vs Number of Iterations")
plt.legend()
plt.grid(False)
plt.show()
```

Improving Model Accuracy for Standardized Data

```
[ ] ckd_df = pd.DataFrame(ckd_X, columns=[f'feature_{i}' for i in range(ckd_X.shape[1])])
ckd_y_df = pd.DataFrame(ckd_y, columns=[f'target'])
df = pd.concat([ckd_X_df, ckd_y_df], axis=1)

corr_matrix = df.corr().abs()
threshold = 0.6 # or above, doesn't change accuracy value
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))

# features to drop
to_drop = [column for column in upper_tri.columns if any(upper_tri[column] > threshold)]
ckd_df_drop = df.drop(columns=to_drop)

train_ckd_x = ckd_df_drop.iloc[:, :-1].to_numpy()
train_ckd_y = ckd_df_drop.iloc[:, -1].to_numpy()

# train and validate model
model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
accuracy = k_fold_validation(model, train_ckd_x, train_ckd_y)

print("CKD accuracy after removing subset of features:", accuracy)

bat_X_df = pd.DataFrame(battery_X, columns=[f'feature_{i}' for i in range(battery_X.shape[1])])
bat_y_df = pd.DataFrame(battery_y, columns=[f'target'])
df = pd.concat([bat_X_df, bat_y_df], axis=1)

corr_matrix = df.corr().abs()
threshold = 0.7 # or above
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))

# features to drop
to_drop = [column for column in upper_tri.columns if any(upper_tri[column] > threshold)]
bat_df_drop = df.drop(columns=to_drop)

train_bat_x = bat_df_drop.iloc[:, :-1].to_numpy()
train_bat_y = bat_df_drop.iloc[:, -1].to_numpy()

# train and validate model
model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
accuracy = k_fold_validation(model, train_bat_x, train_bat_y)

print("Battery accuracy after removing subset of features:", accuracy)
```

```

print("Squaring features: ")
for x in range(3):
    index = x + 1
    train_ckd_x = np.hstack((ckd_X, ckd_X[:, index:index+1] ** 2))
    train_bat_x = np.hstack((battery_X, battery_X[:, index:index+1] ** 2))

    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    accuracy1 = k_fold_validation(model, train_ckd_x, ckd_y)
    accuracy2 = k_fold_validation(model, train_bat_x, battery_y)
    print(index, f'CKD: {accuracy1:.4f}', f'Battery: {accuracy2:.4f}')

print("Cubing features: ")
for x in range(3):
    index = x + 1
    train_ckd_x = np.hstack((ckd_X, ckd_X[:, index:index+1] ** 3))
    train_bat_x = np.hstack((battery_X, battery_X[:, index:index+1] ** 3))

    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    accuracy1 = k_fold_validation(model, train_ckd_x, ckd_y)
    accuracy2 = k_fold_validation(model, train_bat_x, battery_y)
    print(index, f'CKD: {accuracy1:.4f}', f'Battery: {accuracy2:.4f}')

epsilon = 1e-5 # add to functions to avoid zeroes

print("Log of features: ")
for x in range(3):
    index = x + 1
    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    train_log = np.log(ckd_X[:, index:index+1])
    train_ckd_x = np.hstack((ckd_X, train_log))
    accuracy1 = k_fold_validation(model, train_ckd_x, ckd_y)

    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    train_log = np.log(battery_X[:, index:index+1] + epsilon)
    train_bat_x = np.hstack((battery_X, train_log))
    accuracy2 = k_fold_validation(model, train_bat_x, battery_y)

    print(index, f'CKD: {accuracy1:.4f}', f'Battery: {accuracy2:.4f}')

```

```

print("Square root of features: ")
for x in range(3):
    index = x + 1
    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    train_sqrt = np.sqrt(ckd_X[:, index:index+1])
    train_ckd_x = np.hstack((ckd_X, train_sqrt))
    accuracy1 = k_fold_validation(model, train_ckd_x, ckd_y)

    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    train_sqrt = np.sqrt(battery_X[:, index:index+1] + epsilon)
    train_bat_x = np.hstack((battery_X, train_sqrt))
    accuracy2 = k_fold_validation(model, train_bat_x, battery_y)

    print(index, f'CKD: {accuracy1:.4f}', f'Battery: {accuracy2:.4f}')

print("Reciprocal of features: ")
for x in range(3):
    index = x + 1
    train_ckd_x = np.hstack((ckd_X, 1 / (ckd_X[:, index:index+1] + epsilon)))
    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    accuracy1 = k_fold_validation(model, train_ckd_x, ckd_y)

    train_bat_x = np.hstack((battery_X, 1 / (battery_X[:, index:index+1] + epsilon)))
    accuracy2 = k_fold_validation(model, train_bat_x, battery_y)

    print(index, f'CKD: {accuracy1:.4f}', f'Battery: {accuracy2:.4f}')

print("Multiplication of pairs of features: ")
for x in range(3):
    index = x + 1
    train_ckd_x = np.hstack((ckd_X, ckd_X[:, index:index+1] * ckd_X[:, index+1:index+2]))
    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    accuracy1 = k_fold_validation(model, train_ckd_x, ckd_y)

    train_bat_x = np.hstack((battery_X, battery_X[:, index:index+1] * battery_X[:, index+1:index+2]))
    accuracy2 = k_fold_validation(model, train_bat_x, battery_y)

    print(index, f'CKD: {accuracy1:.4f}', f'Battery: {accuracy2:.4f}')

```

```

print("Sin of features: ")
for x in range(3):
    index = x + 1
    train_ckd_x = np.hstack((ckd_X, np.sin(ckd_X[:, index:index+1])))
    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    accuracy1 = k_fold_validation(model, train_ckd_x, ckd_y)

    train_bat_x = np.hstack((battery_X, np.sin(battery_X[:, index:index+1])))
    accuracy2 = k_fold_validation(model, train_bat_x, battery_y)

    print(index, f'CKD: {accuracy1:.4f}', f'Battery: {accuracy2:.4f}')

```

```

# graphing logarithm accuracies
ckd_accuracies = []
battery_accuracies = []

epsilon = 1e-5

for x in range(28):
    index = x + 1

    if (ckd_X[:, index:index+1] + epsilon < 0).any():
        break
    if (battery_X[:, index:index+1] + epsilon < 0).any():
        break

    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    train_log = np.log(ckd_X[:, index:index+1] + epsilon)
    train_ckd_x = np.hstack((ckd_X, train_log))
    accuracy1 = k_fold_validation(model, train_ckd_x, ckd_y)

    model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
    train_log = np.log(battery_X[:, index:index+1] + epsilon)
    train_bat_x = np.hstack((battery_X, train_log))
    accuracy2 = k_fold_validation(model, train_bat_x, battery_y)

    ckd_accuracies.append(accuracy1)
    battery_accuracies.append(accuracy2)

    print(index, f'CKD: {accuracy1:.4f}', f'Battery: {accuracy2:.4f}')

# Plot results
plt.figure(figsize=(10, 6))
plt.plot(ckd_accuracies, marker='o', label="CKD Dataset")
plt.plot(battery_accuracies, marker='s', label="Battery Dataset")

plt.xlabel("Index for Feature Logarithm")
plt.ylabel("Accuracy")
plt.title("Model Accuracy vs Index")
plt.legend()
plt.grid(False)
plt.show()

```

```

# Accuracy of different models given the same learning rate and epoch
default_model = LogisticRegression("default", learning_rate=0.3, num_epochs=1000)
l1_model = LogisticRegression("l1", learning_rate=0.3, num_epochs=1000)
l2_model = LogisticRegression("l2", learning_rate=0.3, num_epochs=1000)
models = [default_model, l1_model, l2_model]

for model in models:
    accuracy1 = k_fold_validation(model, ckd_X, ckd_y)
    accuracy2 = k_fold_validation(model, battery_X, battery_y)
    print(model.regularization, f'CKD: {accuracy1:.4f}', f'Battery: {accuracy2:.4f}')

```

```

# graphs accuracy over range of k's
iterations = [5, 6, 7, 8, 10, 15]
ckd_accuracies = []
battery_accuracies = []

for test_k in iterations:
    model = LogisticRegression(learning_rate=0.3, num_epochs=1000)

    ckd_accuracy = k_fold_validation(model, ckd_X, ckd_y, k = test_k)
    battery_accuracy = k_fold_validation(model, battery_X, battery_y, k = test_k)

    ckd_accuracies.append(ckd_accuracy)
    battery_accuracies.append(battery_accuracy)

# Plot results
plt.figure(figsize=(10, 6))
plt.plot(iterations, ckd_accuracies, marker='o', label="CKD Dataset")
plt.plot(iterations, battery_accuracies, marker='s', label="Battery Dataset")

plt.xlabel("Value of K")
plt.ylabel("Accuracy")
plt.title("Model Accuracy vs K Value")
plt.legend()
plt.grid(False)
plt.show()

```